The Substitution State Machine: Monotonicity, Monotone Reads, Clone Stability, and Rollback

Deep dive E for "Refinement and Inference Growth in SKJS"

Models revision `cf5a6bc7` of `main` (2026-07-09)

Abstract

The main paper models the inference substitution as a state machine; four of its ledger rows concern that machine's state discipline — axiom **L-MONO** (refinement-time monotonicity), the obligation *Monotone Reads Under Growth*, assumption **L-CLONE** (clone stability), and the obligation *No Commitment on Failure* — and this paper is the evidence behind their current table statuses. This deep dive completes the state model the main paper elides — the version counter, the four caches, the environment-side cycle sets, constraint attachment, and the candidate-freezing flag — and works the four rows as one coherent discipline. The load-bearing discovery is representational: `TVar::setConstraint` is a *functional update* on a frozen value, not a mutation of shared state, which removes the envelope-shrinking threat the main paper flagged for **L-MONO** and **L-CLONE**. With that, **L-MONO** is proved under two stated modeling conditions; *Monotone Reads* is proved by a read-variance audit everywhere except one VIOLATED residue, MR-1 — a freeze-window unsealed read memoized in `seenSub`, witnessed end-to-end by an order-dependent unsound acceptance (issue draft `issues/freeze-window-stale-memo-caches.md`); **L-CLONE** is proved conditional on a strengthened, fork-state form of **L-INTER** — the clone-generated-identifier hazard is verified *unreachable* (no `gen*` call site lies in the clone-compute-install tree), guarded by a recommended CI assertion; and *No Commitment on Failure* is discharged by an exhaustive save/restore audit with one intentional, separately-argued exception (array-literal form commitments persisting on failed checks). Failed proof attempts are reported as findings.

Contents

1 Purpose and Rows Addressed **2**

2 The Complete State Model **2**

2.1 Saved state: `SubstState` . . . 2
2.2 Unsaved state . . . 3
2.3 Constraint attachment is a value operation . . . 4
2.4 The extended transition table . . . 4

3 The Order-Theoretic Core **5**

4 L-MONO: Refinement-Time State Monotonicity **6**

4.1 Enumeration of refinement-triggered evolutions . . . 6
4.2 Discharge . . . 7

5 Monotone Reads Under Growth **8**
5.1 The invariant 8
5.2 Site audit 8
5.3 Residual hazard: the freeze-window unsealed read 9

6 L-CLONE: Clone–Compute–Install **10**
6.1 The pattern and what is actually lost 10
6.2 Over-approximation: the positive result 10
6.3 Clone-escaped fresh identifiers: an unreachable hazard 11

7 No Commitment on Failure: the Rollback Audit **11**
7.1 Exhaustive save/restore pairing 12
7.2 The intentional exception: array-literal commitments 12
7.3 `tenv.seenSub`: the #446 argument, made precise 13

8 Status Table and Ledger Edits **14**

A Implementation Anchors **16**

1 Purpose and Rows Addressed

This document supports the "Assumptions and Proof Obligations" ledger of the main paper. It addresses four rows — **L-MONO**, Monotone Reads Under Growth, **L-CLONE**, and No Commitment on Failure — treated in Sections 4, 5, 6, and 7, and summarized row by row in §8.

All file and line references are against commit `cf5a6bc7`. Line numbers are cited as `file:line`; they identify the revision precisely and will drift with edits.

2 The Complete State Model

The main paper's Definition "Substitution States" records only the binding map and the freeze flag. The implementation state is larger. This section is the reference model; everything later in the paper is proved against it.

2.1 Saved state: `SubstState`

Definition 1 (Full Substitution State)**.** *A substitution state ρ consists of the fields of the Skip class `SubstState` (`TTypeSubst.sk:13-49`):*

- values : TVarID ⇀ SVOpen(TType) | SVClosed(TType) — *the binding map of the main paper's transition table;*
- version ∈ ℕ — *a counter with two duties: (a) it is incremented immediately before* every *write to* values *(`grow`, `TTypeSubst.sk:181`; `get`'s two sealing writes, `:195` and `:205`); and (b) it doubles as the fresh-identifier supply — `gen`, `genForField`, and `genForLiteralCommitment` (`:237-253`) each return `TVarID(version)` and increment. Consequence: within one linear execution segment, distinct* values *states have distinct versions (Lemma 9);*
- origins — *diagnostic provenance for variables (`setOrigin`); no semantic content;*
- *four caches, all part of the saved state:*

– seenSub $\subseteq$ TType $\times$ TType — *a* success memo *for* ***TType::isSub****: a pair is inserted on acceptance (****TTypeSubtyping.sk:1591****) and consulted before re-checking (****:1588****). Not version-keyed;*

– seenInter, seenUnion : (TType $\times$ TType) $\rightharpoonup$ TType — *result memos for* ***TType::inter*** *(****TTypeInter.sk:107-125****; pairs containing a bare variable are excluded,* ***:105****) and* ***TType::union*** *(****TTypeUnion.sk:29-32****);*

– seenExpansion : TSInst $\rightharpoonup$ TType — *memo of one-level named-type expansion (****TType.sk:650-653****); the cached body is* symbolic *in the type arguments (arguments occur as free variables, re-resolved at every use), which is why reuse under growth is sound (comment* ***TTypeSubst.sk:17-30****).*

save() *snapshots all seven fields into a frozen* ***SavedSubst****;* ***restore()*** *reinstates all seven (****TTypeSubst.sk:36-48****). Because every field is an immutable persistent map/set,* ***save*** *is* $O(1)$ *and lossless.*

impl: ***TTypeSubst.sk / SubstState, SavedSubst***

2.2 Unsaved state

Three pieces of mutable or threaded state live *outside* `SubstState`; each needs its own safety argument.

(U1) `freezeCandidates`**.** Declared on `Subst` itself, not on `SubstState` (`TTypeSubst.sk:76-89`), with an explicit comment that save/restore never touch it; it is push/popped around the constraint-admissibility check in `growTVars` (`TTypeSubtyping.sk:1431-1443`). Verified by inspection: those two lines are the *only* writes to the flag in `src/`, the push/pop brackets a straight-line region with no early return (the `break` at `:1440` exits only the inner constraint loop), and nested checks restore the outer value via `wasFreezing`. So excluding it from `SavedSubst` is safe: a restore can never observe a torn flag, because every save/restore pair either lies entirely inside or entirely outside a freeze window.

(U2) `tenv.seenSub` **and** `tenv.seenInter`**.** `TEnv` is a *frozen* Skip class (`TEnv.sk:12`); the apparent mutations `!tenv.seenSub = ...` (`TTypeSubtyping.sk:1537`) and `!tenv.seenInter = ...` (`TTypeInter.sk:157`) are functional updates of the *local* binding, propagated only into callees that receive the updated `tenv`. These sets are therefore *path-scoped coinduction hypotheses*, not caches: an entry is visible exactly in the dynamic extent of the recording activation and vanishes when it returns. This structural fact does most of the work in the #446 argument (§7.3). `tenv.seenSub` is keyed by (version, u_1, u_2) (`TEnv.sk:74`); `tenv.seenInter` by the bare pair (`TEnv.sk:69`), which is safe because its only effect is to return an *unexpanded* intersection on a cycle (`TTypeInter.sk:154-156`) — a representation choice, not a judgement.

(U4) `tenv.expandInter`**.** A mode flag on `TEnv` selecting full versus deferred (`unexpandedInter`) field intersection; set by `TInter::expand` and, like `tenv.seenSub`, threaded through the frozen-`TEnv` update discipline, so the setting should be path-scoped. This flag was found missing from this inventory by external review; its path-scoping and the containment coverage of the deferred form are the OPEN registry row TENV-EXPANDINTER.

(U3) `results` and `context`. The diagnostics/callgraph accumulator `results` is shared by `clone()` (`TTypeSubst.sk:137-149`) and never rolled back (e.g. `recordSubtypingEdge` on `TFun::isSub` success, `TTypeSubtyping.sk:1681-1688`). It carries no typing state, so it is outside ρ; §6 flags the one place this is worth a remark.

2.3 Constraint attachment is a value operation

A natural worry about the main paper's Axiom **L-MONO** would be that adding a constraint to a *shared* variable (`TVar::setConstraint`) shrinks that variable's envelope, so an arbitrary type mentioning it could lose members. The premise of that worry is false — establishing this is the single most consequential fact in this paper, and the axiom's remark in the main paper records it.

Lemma 2 (setConstraint is Functional)**.** *`TVar::setConstraint(ty)` does not modify any state observable by another type term. It returns a* new *`TVar` value carrying the enlarged constraint set; every pre-existing occurrence of the same variable identifier retains its own constraint set.*

Proof (by inspection of the pinned source). `TVar` is a frozen (non-`mutable`) class and `setConstraint` a non-`mutable` method (`TType.sk:2589` and `:2590-2591`):

```
fun setConstraint(ty: TType): TVar {
  !this.constraints = this.constraints.set(ty); this }
```

In Skip, field assignment through `!this.f = e` in a frozen-class method is a *functional record update of the local binding* `this`; the method must return the updated copy for the change to be visible at all. The same idiom appears throughout the pinned revision on unambiguously frozen classes (`STErrorDetail::ensureIntermediateDepth`, `TTypeSubtyping.sk:40-45`; `TEnv::instantiate`, `TEnv.sk:131`). Two corroborations: (i) `constraints` participates in `TVar::xxhash` (`TType.sk:2598-2607`) and in `Orderable` equality — a shared-mutation semantics would break hashing of interned type terms; (ii) both call sites use the return value (`TType::var(tvar.setConstraint(ty2))`, `TTypeInter.sk:145-151`; `!tvar = tvar.setConstraint(...)`, `TEnv.sk:314`) — a genuine mutation would make that redundant. □

Corollary 3 (Occurrence-Relative Envelopes)**.** *The "constraint intersection" step of `Subst::get` (`TTypeSubst.sk:218-221`) intersects the binding with the constraints* of the occurrence being read. *A variable identifier therefore has no global constraint set; constraints are part of the type term. Refinement, intersection, or any other operation attaching a constraint produces a new term and cannot shrink the denotation of any type already installed in an environment. The state ρ of Definition 1 is the* complete *shared mutable typing state.*

2.4 The extended transition table

Definition 4 (Extended Transitions)**.** *The operations that evolve ρ, extending the main paper's Definition 6 (its rows are unchanged and not repeated):*

op	*effect on* ρ	*anchor*
grow, get(cov) *writes*	*per main paper;* version++ *before each write*	`TTypeSubst.sk:151-222`
gen/genFor∗	*fresh id =* version*;* version++*; no binding*	`TTypeSubst.sk:237-253`
cache insert	*add to one of the four memo maps; no version bump*	*§1*
setOrigin	*origins only*	`TTypeSubst.sk:271`
save/restore	*snapshot / reinstate all seven fields*	`TTypeSubst.sk:36-48`
clone	*fork: new* `Subst`*, same seven fields, shared* `results`	`TTypeSubst.sk:137-149`
freeze push/pop	*toggle* `freezeCandidates` *(unsaved)*	`TTypeSubtyping.sk:1431-1443`

Verified write-site completeness: the only assignments to `values` *in* `src/` *are in* `grow`*,* `get`*, and* `restore`*; the only version writes are those plus* `gen*`*; external* `grow` *callers (*`TArrayLitTyping.sk:147,352`*;* `TVariadicTyping.sk:125`*;* `TypeCheckUtils.sk:310,321,332`*) go through the same transitions.*

Note that cache inserts do *not* bump the version. This is sound for the three result memos by their own stability arguments (sealing for `seenInter`, `TTypeInter.sk:90-96`; symbolic arguments for `seenExpansion`) but it makes the version-keying of `tenv.seenSub` track *binding* state only — exactly what the coinduction hypothesis depends on (§7.3).

3 The Order-Theoretic Core

All four ledger rows reduce to statements about one partial order on states and two lemmas about it.

Definition 5 (Envelope; State Order)**.** *For a state* ρ *and identifier* α*, the* envelope *is*

$$\mathcal{E}_\rho(\alpha) = \begin{cases} \emptyset & \alpha \notin \mathrm{dom}(\rho.\mathsf{values}) \\ [\![t]\!]_\rho & \rho.\mathsf{values}(\alpha) \in \{\mathsf{SVOpen}(t), \mathsf{SVClosed}(t)\}. \end{cases}$$

Write $\rho \sqsubseteq \rho'$ *("*ρ' *extends* ρ*") iff* $\mathcal{E}_\rho(\alpha) \subseteq \mathcal{E}_{\rho'}(\alpha)$ *for every* α*. The choice* $\emptyset$ *for unbound variables is the modeling counterpart of* get(contra) *returning* $\bot$ *without writing (*`TTypeSubst.sk:211-212`*).*

Definition 6 (Modeling Conditions)**.** ***(M1)*** Semantic monotonicity. *For every type term* u*,* $[\![u]\!]_\rho$ *is monotone in the envelopes of the identifiers occurring in* u*:* $\rho \sqsubseteq \rho'$ *implies* $[\![u]\!]_\rho \subseteq [\![u]\!]_{\rho'}$*. This holds for the main paper's semantics by structural induction, because every state-sensitive position is read positively: variable occurrences denote* $\mathcal{E}_\rho(\alpha) \cap \bigcap [\![\mathrm{cstr}]\!]$ *(Corollary 3); field membership constrains only the* get *type (main paper, object semantics, which makes set types opaque); call/new signatures are opaque to* $[\![\cdot]\!]$ *except for the callability mode; and union/intersection/clause structure is positive. The classic contravariance counterexample is deflected by signature opacity, exactly as the main paper notes for* ***L-MONO****.*

(M2) Union monotonicity. ***L-UNION*** *at the point of use:* $[\![t_2]\!]_\rho \subseteq [\![t_2 \vee t]\!]_\rho$*.*

Lemma 7 (All Transitions Are Monotone)**.** *Under M2, every operation of Definition 4 except* `restore` *takes* ρ *to some* ρ' *with* $\rho \sqsubseteq \rho'$*.*

Proof. Case analysis on the main paper's transition table plus Definition 4. Fresh bind (unbound → $\mathsf{SVOpen}(t)$ or $\mathsf{SVClosed}(t)$): envelope grows from $\emptyset$. Fresh covariant get: unbound → $\mathsf{SVClosed}(\top)$: grows to $[\![\top]\!]$. Open union ($\mathsf{SVOpen}(t_2) \to \mathsf{SVOpen}(t_2 \vee t)$, or $\mathsf{SVClosed}(t_2 \vee t)$ for non-candidates): grows by M2. Seal ($\mathsf{SVOpen}(t) \to \mathsf{SVClosed}(t)$): envelope unchanged. No-op rows, gen (the new

identifier is unbound before and after — its id was never allocated before, so no term mentions it), cache inserts, `setOrigin`, freeze toggles: envelopes unchanged. No transition removes or replaces bound material: there is no row that shrinks a bound, and the main paper's never-widen lemma additionally freezes SVClosed rows. Envelopes of *other* variables are untouched by each write (values is updated pointwise), though their envelopes may grow transitively via M1 when their bound mentions α — still monotone. □

Lemma 8 (Restore Safety)**.** *Call a computation segment* well-bracketed *if every `restore(s)` it executes uses a snapshot `s` taken by the same activation (verified for all fourteen sites in §7). Let a well-bracketed segment start at ρ_0 and execute monotone transitions and restores. Then at every moment, $\rho_0 \sqsubseteq \rho_{\text{now}}$.*

Proof. Induction over the event sequence. Monotone steps preserve the invariant by transitivity. A restore jumps to a snapshot taken earlier *within the segment* (well-bracketing), and the invariant held when the snapshot was taken, so it holds after the jump. □

Lemma 9 (Record-Point Coherence)**.** *Let a moment m_0 (the* record point*) occur at the* entry *of an activation A, with $\rho.\mathsf{version} = v$ at m_0, and consider any later moment m inside A's dynamic extent, under well-bracketing. Then (i) $\rho.\mathsf{version} \geq v$ at m, and (ii) if $\rho.\mathsf{version} = v$ at m, then $\rho.\mathsf{values} = \rho.\mathsf{values}$ at m_0.*

Proof. (i) The version increases at writes and `gen`, and decreases only at restores. Under well-bracketing, every restore executed inside A's extent uses a snapshot saved inside A's extent, hence saved *after* m_0 (m_0 is A's entry), hence with version $\geq v$. So no reachable restore target has version $< v$. (ii) By induction on the events between m_0 and m. All version-v snapshots taken in the extent are taken at moments where, by the induction hypothesis, `values` equals its m_0 state (a change since m_0 would have bumped the version above v, and by (i) it cannot have come back down except by restoring a version-v snapshot, itself coherent by induction). A moment with version v is either reached with no intervening `values` write since the last coherent version-v moment, or by restoring a coherent version-v snapshot. □

Finding 10 (Failed proof attempt, kept for honesty)**.** *The naive claim "`version` uniquely identifies `values` across a whole checking run" is* false*: restore rewinds the counter, so divergent exploration branches can reuse a version number with different bindings (save at v, grow to $v{+}1$, restore, grow differently to $v{+}1$). Any argument that treats the version as a global state hash — including a reading of the #446 fix as one — is unsound. The actual safety of `tenv.seenSub` comes from the* combination *of version-keying with path-scoping, Lemma 19 below.*

4 L-MONO: Refinement-Time State Monotonicity

L-MONO (the main paper's axiom of that name) demands: for every evolution $\rho \to \rho'$ performed while refining a condition, and every $u \in \text{range}(\Gamma)$, $v \in [\![u]\!]_\rho \implies v \in [\![u]\!]_{\rho'}$.

4.1 Enumeration of refinement-triggered evolutions

By a walk of every arm of `evalCond` (`TypeCheckRefine.sk:371-541`) and its handlers, the evolutions of the *main* substitution that condition refinement can trigger are exactly:

(E1) *Covariant expansion in the refinement primitive.* `setLocalFilterMap` runs `mapSTypes(..., Some(Covariant()), f)` on the main substitution (`TypeCheckRefine.sk:615`); expansion applies `subst.get(tenv, tvar, Covariant())` to every spine variable (`TType.sk:660`): fresh variables seal to $\top$, open variables seal at their bound. Used by *every* rule that factors through the primitive.

(E2) *Union/intersection during residue and clause rebuilding.* `mapSTypes` reassembles clauses with `Clauses::unionN` (`TType.sk:411`); the `typeof == "object"` default builds `STypes::union` on the main substitution (`TypeCheckRefine.sk:273-285`). Union itself performs no `get`/`grow` (verified: no such call in `TTypeUnion.sk`) but recursively calls `TType::inter` on field set-types (`TTypeUnion.sk:102,124`), reaching E3.

(E3) *Main-substitution intersections.* `evalIsArray`'s true branch and `evalInstanceof` run `TObject::inter` on the main substitution (`TypeCheckRefine.sk:334,361`). Inside `TType::inter_`: covariant expansion of both operands (`TTypeInter.sk:158-159`, seals as in E1); constraint attachment on bare-variable operands (`TTypeInter.sk:145-151`, a value operation by Lemma 2); speculative concrete `isSub` probes (`:163-188`); and `TFun::inter`'s two `TFun::isSub` attempts on the main substitution (`TTypeInter.sk:602, 606`) — on success their `growTVars` growth *commits*, on failure it is restored.

(E4) *Fresh variable generation.* `evalInstanceof` instantiates the class with fresh inference arguments (`makeTArgs`, `TEnv.sk:308-323`: `gen` + occurrence-level `setConstraint` + `setOrigin`); `newObject` may create synthetic index variables (`TType.sk:1764-1780`).

(E5) *Cache inserts* in all of the above.

Everything else refinement does runs on a *clone* (`TypeCheckRefine.sk:89,99,177,197,573`) and does not evolve the main ρ at all; those sites are §6's subject.

4.2 Discharge

Theorem 11 (L-MONO)**.** *Under M1 and M2 (Definition 6), every evolution E1–E5 satisfies the* ***L-MONO*** *conclusion — in fact for* all *type terms u, not only* range(Γ).

Proof. E1–E5 are compositions of the transitions of Definition 4 and internal save/restore pairs (inside the speculative `isSub` calls of E3). All fourteen save/restore sites are well-bracketed (§7), so Lemma 8 applies to the whole composition: $\rho \sqsubseteq \rho'$. M1 turns this into $[\![u]\!]_\rho \subseteq [\![u]\!]_{\rho'}$. Two of the main paper's worries dissolve: (a) the `setConstraint` threat is void by Lemma 2 — no shared envelope shrinks, so the restriction to range(Γ) is not needed; (b) the E3 commits from a *successful* speculative `TFun::isSub` are ordinary monotone grows, so "exploration that succeeds and keeps its growth" is safe, not merely "exploration that fails and rolls back." □

Finding 12 (The sharpest sufficient condition is not needed)**.** *The task of bounding* which *variables refinement may touch ("refinement only constrains variables introduced by the very expression being refined") was pursued and abandoned: it is both false — `evalIsArray`/`evalInstanceof` intersect against $\Gamma(x)$'s own object branches, sealing variables shared with the whole definition's checking — and unnecessary, since sealing and growing are monotone for* every *sharer. The variable-sharing pattern is irrelevant once Lemma 2 removes the only non-monotone candidate operation.*

Residual dependence. Theorem 11 is a genuine discharge of **L-MONO** *given the main paper's semantics*: M1 is a property of that semantics (provable by structural induction, sketched in Definition 6; it deserves a short formal proof when the semantics section is next revised), and M2 is **L-UNION**, already a ledger row. The ledger edit in §8 states this precisely.

5 Monotone Reads Under Growth

The obligation: a judgement validated under ρ must still hold after inference evolves $\rho \to \rho'$. For SVClosed bindings this is never-widen(iii); the open question is SVOpen bindings, read without sealing by get(contra).

5.1 The invariant

Definition 13 (Read Disciplines)***(MR)*** Monotone-read invariant. *Every contravariant (non-sealing) read is a* supertype-side *read: its result is used only as the right-hand side of a validation* $[\![\text{lhs}]\!] \subseteq [\![\cdot]\!]$*, so that later growth of the variable only enlarges the right-hand side.*

(BR) Bake-requires-seal. *Every covariant read whose result is embedded in a persistent artifact (a memoized result, a type installed in an environment) seals the variable, so the baked snapshot equals the envelope at all later states (never-widen(iii)).*

Theorem 14 (Validated Judgements Are Stable)**.** *Assume M1, MR, BR, and* ***L-SUB****. If* `Clauses::isSub` *accepted* $[\![u_1]\!]_\rho \subseteq [\![u_2]\!]_\rho$ *and* $\rho \to \rho'$ *by the transitions of Definition 4 (no restore crossing the acceptance), then* $[\![u_1]\!]_{\rho'} \subseteq [\![u_2]\!]_{\rho'}$*.*

Proof. Decompose the occurrences of variables in u_1, u_2 by how validation read them. Left-side occurrences were read covariantly and are sealed on the success path (BR); never-widen(iii) freezes their envelopes, so $[\![u_1]\!]$ is unchanged... more precisely non-increasing only through frozen envelopes, hence unchanged. Right-side occurrences were read contravariantly (MR); their envelopes only grow (Lemma 7), and by M1 $[\![u_2]\!]$ only grows. The accepted inclusion is therefore preserved. □

5.2 Site audit

Every `expandToClauses`/`expand`/`get` site with its variance, in the pinned revision. Expansion applies its variance uniformly to every *spine* variable, including through `TSInst` and `TInter` unfolding (`TType.sk:634-678`); it does not enter fields or signatures, so no variance flip arises inside a single expansion — flips happen only where the recursion of `isSub` swaps sides.

site	var.	discipline check
`isSub_` lhs u_1 (TTypeSubtyping.sk:1623)	cov	BR: seals; memo `seenSub` baked on success
`isSub_` rhs u_2 (:1624)	contra	MR: supertype side
`TFun::isSub_` rest, *sub* side (:1708)	contra	MR: becomes rhs of param check `isSub(p2, p1)`
`TFun::isSub_` rest, *super* side (:1768–1773)	cov	BR: lhs of param checks; sealed
`TTypeInter` both operands (TTypeInter.sk:158–159)	cov	BR: result memoized/installed; seal justifies `seenInter`
`TypeCheckUtils` truthiness reads (:19,43)	cov	BR: result drives diagnostics/refinement
`resolveSpreadToTuple` (TVariadicTyping.sk:53,74)	contra	MR: tuple shape probe; restored when `None` (:64,70,78)
TEnv.sk:820,2103; TypeCheckClass.sk:55,146; TypeCheckExpr.sk:1549–1553	cov	BR
`expandTVarsForPp` (TType.sk:703)	cov	on a *clone* (TTypePp.sk:815, TTypeSubtyping.sk:1195–1196): pretty-printing must not seal
`ArrayLitTyping` probes (TArrayLitTyping.sk:175,236,394)	cov	:236 clones; :175,394 seal on the live subst (commitment sites)

No site violates MR: all contravariant reads end up on the right of a validation. BR holds at all sites on the success path, with one exception, which is the section's finding.

5.3 Residual hazard: the freeze-window unsealed read

get(cov) on an open *candidate* under `freezeCandidates` returns the bound *without sealing* (`TTypeSubst.sk:202-203`) — a covariant read that violates BR by design, so that constraint admissibility checks are queries. The check runs full `TType::isSub(boundTy, cstr)` (`TTypeSubtyping.sk:1435`), whose *successes* insert pairs into the persistent, version-*unkeyed* `subst.seenSub` memo.

Finding 15 (Stale `seenSub` memo via freeze-window reads)**.** *Candidate scenario: a variable α (an ordinary candidate,* SVOpen(t)*) occurs in* both *sides of a check — e.g. $u_1 = \mathrm{Box}\langle\alpha\rangle$ and u_2 mentions a constrained variable whose constraint check validates* isSub(u_1, cstr) *under freeze, reading α at t* unsealed *and memoizing* $(u_1, \mathrm{cstr}) \in$ seenSub*. After the freeze pops,* `growUncovered` *may contravariantly grow the same α (it is no longer protected) to $t \vee s$. The memo now asserts an inclusion validated with the smaller $[\![u_1]\!]$; a later syntactically-identical query hits it (`TTypeSubtyping.sk:1588`) without re-validation. The eventual covariant expansion of u_1 seals α at $t \vee s$ —* after *the memo was recorded. The discipline itself cannot close this gap: the memo survives on the enclosing check's success path, and nothing re-keys it by version. Mitigations narrow the window without closing it (memos recorded under a speculative subtree are erased by its restore, since* seenSub *is in `SubstState`; the scenario needs the same open candidate reachable from both sides plus an exact term-level re-query). The gap is real: an end-to-end witness exists — an order-dependent unsound acceptance in which* adding *an earlier call to a generic function (validating its constraint under freeze against the still-open candidate, and memoizing the success) flips a correct contravariance rejection of the post-widening call into an acceptance, so a `number`-typed binding holds `"s"` at runtime. MR-1 (§8)*

is therefore VIOLATED. *Fix: key* `subst.seenSub` *by version, or suppress memo insertion while* `freezeCandidates` *is set; the same fix should cover* `seenInter`*, which the witness also exercises. Issue draft:* `issues/freeze-window-stale-memo-caches.md`*; a pinned test is recommended.*

Theorem 14 discharges the obligation *conditional on* MR-1 and on **L-SUB**, and the audit above discharges its MR/BR premises for every other site; since MR-1 fails, the obligation stands VIOLATED exactly at the freeze-window memo and PROVED everywhere else.

6 L-CLONE: Clone–Compute–Install

6.1 The pattern and what is actually lost

`evalVarEquality` computes $u = \mathsf{inter}(\Gamma(x), \tau_e)$ under `subst.clone()` and installs u into the scope, discarding the clone (`TypeCheckRefine.sk:87-94`); likewise `removeConstant` (`:96-106`), `evalFieldEqualityType` (`:176-201`), and `evalFieldTruthiness` (`:571-576`). `clone()` forks all seven `SubstState` fields (immutable maps, so the fork is a true snapshot) and shares only `results` and `context` (`TTypeSubst.sk:137-149`).

Write ρ_f for the fork state (= the main state at the clone), ρ_c for the clone state when `inter` returns, and ρ'' for any later main state. What the clone loses at discard, classified by what `inter` can do:

(C1) *Constraint attachments* (`TTypeInter.sk:145-151`). By Lemma 2 these never touched the clone's state at all: the constraint lives in the returned occurrence inside u, which is installed. *Nothing is lost.* A natural conjecture ("if constraints live in the returned type, clone-discard may be benign for constraints but not for value bindings") is confirmed on its first half.

(C2) *Seals* from covariant expansion (`TTypeInter.sk:158-159`): the expansion replaces the spine occurrence by the read value in u — fresh variables bake $\top$, open variables bake their fork-state bound t — but the seal itself dies with the clone; on the main substitution the variable stays open/unbound and may later grow or seal differently.

(C3) *Grows and commitments* from speculative subtyping inside `inter`: `TFun::inter` runs `TFun::isSub` both ways on the clone (`TTypeInter.sk:602, 606`), committing growth on success; `ArrayLitTyping` may commit a literal's form variable on the clone. All lost at discard, while u may mention the affected variables.

(C4) *Fresh generations* on the clone (expansion of a cold `TSInst` can reach `tenv.instantiate` and, through class setup, `gen`-sites such as synthetic index fields, `TType.sk:1764-1780`). The clone's version counter advances privately; the main counter does not.

6.2 Over-approximation: the positive result

Theorem 16 (Clone Stability)**.** *Assume M1, M2, and the fork-state form of* ***L-INTER****:*

(I$_f$) $[\![t_1]\!]_{\rho_f} \cap [\![t_2]\!]_{\rho_f} \subseteq [\![u]\!]_{\rho''}$ for every $\rho'' \sqsupseteq \rho_f$*, where* $u = \mathsf{inter}$ *computed by a well-bracketed run from* ρ_f*.*

Then ***L-CLONE*** *holds: the installed u over-approximates the fork-state intersection under the main substitution at install time and at all later states. Moreover I_f follows from the ordinary* ***L-INTER*** *plus M1 for every value whose membership does not depend on clone-side state that u bakes or mentions; the two ways such dependence can arise are exactly C2/C3 occurrences and C4 identifiers, analyzed below.*

Proof sketch. The clone evolves monotonically from ρ_f (Lemma 7 + Lemma 8): $\rho_f \sqsubseteq \rho_c$. The main substitution also evolves monotonically: $\rho_f \sqsubseteq \rho''$. Ordinary **L-INTER** gives containment in $[\![u]\!]_{\rho_c}$; the gap is that ρ_c and ρ'' are *incomparable* (divergent branches of ρ_f), so M1 cannot bridge them directly — this was the first failed proof attempt, and the reason **L-CLONE** cannot be reduced to **L-INTER** + M1 alone. The bridge is per-occurrence: (C2) material *baked* into u is the clone-state envelope of the variable at read time, which by monotonicity of the clone run is $\supseteq$ its fork-state envelope; baked material is state-independent thereafter, so reading u under ρ'' yields at least what reading it with fork-state envelopes would — and the fork-state intersection is contained in that by **L-INTER** *evaluated at* ρ_f. (C1) occurrences carry their constraints with them and denote $\mathcal{E}_{\rho''}(\alpha) \cap \bigcap [\![\text{cstr}]\!]$, which by $\rho_f \sqsubseteq \rho''$ contains the fork-state value — the intersection semantics is exact here. (C3) growth affects u only through signature positions (`TFun::inter` returns one of the two function terms, whose parameter/return types may mention clone-grown variables), and signatures are opaque to $[\![\cdot]\!]$ — the main paper's signature-opacity deflection applies verbatim. Field-get positions reach `inter` recursively and fall under C1/C2. What the sketch does *not* cover is C4, next. □

6.3 Clone-escaped fresh identifiers: an unreachable hazard

Finding 17 (Identifier collision between clone and main is unreachable)**.** *Fresh identifiers are drawn from the* version counter *(`TTypeSubst.sk:237-253`), and a clone forks the counter: if a clone ever generated a variable (C4), its id would equal the id the* main *substitution assigns to its own next generation, and a clone-generated occurrence escaping into the installed u in a semantically visible position (a field get type — synthetic index fields qualify, `TType.sk:1764-1780`) would, once the main substitution binds the colliding id, silently read an unrelated variable's envelope. Nothing in Theorem 16's bridge would cover that: the occurrence's denotation would not be monotone from fork state. The hazard is settled by a reachability audit: of all ten `gen*` call sites,* none *lies in the clone-compute-install call tree (`inter` / `removeConstant` / `expand` / `instantiate` / `normalize` / `newObject` / `addFields`); in particular the two `createSyntheticIndex` generators have exactly two callers, both in the object-literal branch of expression typing, which runs on the* main *substitution — never under a refinement clone. So no clone-side fresh identifier is ever generated during clone-compute-install, no collision can occur, and CL-1 (§8) is* discharged*. The discharge rests on a code invariant that should be guarded: assert (and CI-check) that clone-side runs of `inter` never call `gen*`; drawing ids from a counter shared by reference would remove the hazard class altogether.*

Shared `results`. Clone-side speculative `TFun::isSub` successes record subtyping edges into the shared accumulator even when the exploration is discarded (`TTypeSubtyping.sk:1681-1688`). This does not touch typing state; it can only over-connect the callgraph consumed by later phases. Out of scope for type soundness, recorded here so the boundary is explicit.

7 No Commitment on Failure: the Rollback Audit

The governing contract is stated at the top of the subtyping module (`TTypeSubtyping.sk:4-7`): *every `isSub` function must guarantee that when it returns `STError`, the subst is unchanged.* This section verifies it, pairs every save with its restores, and documents the one place the letter of the contract is violated by design.

7.1 Exhaustive save/restore pairing

All `save`/`restore` sites in the pinned revision (`grep` over `src/`), each verified well-bracketed (restore uses the same activation's snapshot; no early exit escapes the pair):

#	site (save; restore)	restore condition
1	`TFields::isSub` (`TTypeSubtyping.sk:796; 798`)	on `STError`
2	`STypes::isSub` (`:1109; 1136`)	on failure
3	`Clauses::isSub` (`:1149; 1169`)	on overall failure (clause with no matching partner)
4	`TObject::isSub` outer (`:1877; 1889`)	on overall failure
5	`TObject::isSub` per branch pair (`:1881; 1886`)	after each failed pairing, before trying the next
6	`TType::isSub_` (`:1617; 1643`)	on validation failure (growth persists on success, intentionally)
7	`growTVars` per-variable (`:1416; 1447`)	on constraint-check failure
8	`isCovered` probes (`:1354; 1365`)	after *every* probe, hit or miss
9	`TFun::isSub` (`:1676; 1679`)	on `STError`
10	`ArrayLit` tuple commit (`TArrayLitTyping.sk:402; 416`)	on element-check failure (case 2c)
11	`resolveSpreadToTuple` (`TVariadicTyping.sk:51; 64,70,78`)	on every `None` path
12	`inferRestTypeVar` (`TVariadicTyping.sk:124; 130`)	on constraint failure (error also reported)
13	`satisfies` (`TypeCheckExpr.sk:2793; 2805`)	*unconditionally* — `satisfies` must not commit inference
14	clone-as-rollback: `TEnv.sk:222`, `TypeCheckRefine.sk` (5 sites), `TypeCheckExpr.sk` (6 sites), pretty-printer	discard instead of restore

Three structural corollaries. (i) Well-bracketing holds at all sites, so Lemmas 8 and 9 apply globally. (ii) `SavedSubst` captures *all* mutable typing state (Definition 1); the two exclusions are `freezeCandidates` — safe by its own push/pop bracketing, U1 — and `tenv.seenSub` — safe by §7.3. In particular a rollback also erases `seenSub`/`seenInter`/`seenUnion`/`seenExpansion` entries recorded in the failed subtree, so no stale *success* memo survives a rollback. (iii) Because no environment binding is written between any save and its restore (refinement installs happen strictly after the operations return), a restore reinstates a state under which every installed type was already satisfied — rollback needs no analogue of **L-MONO**.

7.2 The intentional exception: array-literal commitments

`ArrayLitTyping::subtyping` runs at the top of `TType::isSub_` *before* the save at `:1617` (`TTypeSubtyping.sk:1612`). Its cases 1 and 2b commit the literal's form variable and then return the result of a subsequent check *even when that check fails* (`TArrayLitTyping.sk:334-341, 368-392`): the commitment persists on an `STError` return, violating the module contract's letter. This is deliberate ("irreversible", `:140`); case 2c by contrast restores the commitment on failure (row 10).

Lemma 18 (The carve-out is sound for accepted programs)**.** *A commitment persisting on a failed check cannot cause unsound acceptance, provided every caller that* discards *an* `STError` *(uses* `isSub` *as a probe) either wraps the call in save/restore or cannot reach an uncommitted array literal.*

Proof, with caller audit. On a non-discarded failure the program is rejected — the committed type may be wrong for the runtime value (case 2b commits to `Array<expected>` even when the elements fail against `expected`), but no judgement about an accepted program uses it. The risk is a discarded failure. Probe callers: `isCovered` (row 8) restores around every probe; `growTVars` constraint checks (row 7) restore on failure; the speculative probes inside `inter_` (`TTypeInter.sk:163-188`) and `SType::overlaps` (`:392-393`) compare a boxed-primitive instance against a single `SType` — neither side is a bare uncommitted `TArrayLit`, which is the only trigger shape (`getSingleArrayLitUncommitted` requires a sole `TArrayLit` inst, `TArrayLitTyping.sk:68-82`); `TFun::inter`'s two attempts run under `TFun::isSub`'s own save/restore (row 9), which was taken *before* the inner `isSub_` ran, hence before the commitment — a failed first attempt rolls the commitment back. The constraint probe at `TEnv.sk:222` runs on a clone (row 14). □

This audit is complete for the pinned revision but fragile — any new `isSub`-as-probe without a save/restore wrapper re-opens it. Sub-obligation RB-1 in §8 records the maintenance condition.

7.3 `tenv.seenSub`: the #446 argument, made precise

`tenv.seenSub` is the coinduction set of `isSub_`: on entry, if $(\mathsf{version}, u_1, u_2)$ is present the check succeeds by coinductive hypothesis; otherwise the triple is recorded and threaded into the recursive calls (`TTypeSubtyping.sk:1603-1604`). It is not part of `SubstState`, and no rollback ever touches it. Its safety rests on two independent mechanisms:

Lemma 19 (Version-Keyed Hypotheses Are Inert or Exact)**.** *A lookup that hits an entry* (v, u_1, u_2) *observes* values *identical to the recorder's state at record time. Consequently a hypothesis is either consulted under exactly the binding state it was made in, or (after any growth, seal, or fresh generation, each of which bumps the version) it is* inert *— present but unmatchable.*

Proof. Two mechanisms. *Path-scoping* (U2, §2): `TEnv` is frozen, so the entry is visible only in the dynamic extent of the recording activation — entries never leak across sibling explorations or survive returns, which already rules out the cross-branch collision of Finding 10. *Record-point coherence*: the record happens at the activation's entry, so Lemma 9 applies to every lookup in the extent: version $= v$ implies `values` equal to record time. (The lemma's premise — restores in the extent use snapshots from the extent — is row-by-row verified in §7.) □

This is the precise content of the #446 regression class and its guard: without the version key, a rolled-back exploration's hypothesis could be consulted under different bindings *within the same extent* (probe loops, rows 7–8, restore and re-grow inside one `isSub_` activation); the version key makes exactly those entries inert. The prerequisite "version is bumped on every `values` write" is verified at all three write sites (`TTypeSubst.sk:181,195,205`); note `gen*` bumps too, which additionally protects hypotheses against fresh-variable aliasing inside an extent.

By contrast, `subst.seenSub` — the success memo, *not* version-keyed — is protected against rollback staleness by living inside `SubstState` (corollary (ii) above), and against growth staleness by Theorem 14; it is the memo that inherits the MR-1 hazard (Finding 15).

8 Status Table and Ledger Edits

Named sub-obligations introduced by this paper:

MR-1 No `subst.seenSub` memo recorded while `freezeCandidates` is set may outlive subsequent growth of an open candidate readable from its left-hand side (Finding 15). Status: VIOLATED, witnessed end-to-end (issue draft `issues/freeze-window-stale-memo-caches.md`). Fix options: version-key the memo, or skip memo insertion under freeze; either must also cover `seenInter`. Pinned test recommended.

CL-1 No fresh variable generated on a clone may occur in a type installed against the main substitution (Finding 17). Status: discharged — the property holds, verified unreachable by the `gen*` call-site audit; guard the underlying invariant with the recommended CI assertion (`gen*` unreachable under clones used by refinement).

RB-1 Maintenance condition: every `isSub` caller that discards `STError` must wrap the call in save/restore or be unreachable by uncommitted array literals (Lemma 18); a CI grep for new probe sites is the cheap enforcement.

M1 Semantic monotonicity of $[\![\cdot]\!]$ in variable envelopes (Definition 6) — a property of the main paper's semantics, provable by structural induction; should be added there as a lemma rather than assumed here.

What this paper establishes, row by row:

Row	Established here
L-MONO	Theorem 11, conditional on M1 and L-UNION; the `setConstraint` threat is void (Lemma 2) and the range(Γ) restriction is droppable
Obl. Monotone Reads	general statement via Theorem 14 (given M1 + the MR/BR audit, §5.2); the residue MR-1 is witnessed (Finding 15)
L-CLONE	Theorem 16 given I_f (fork-state L-INTER; registry row I-F); the id-collision hazard CL-1 discharged as unreachable (Finding 17), CI-assertion guard recommended
Obl. No Commitment on Failure	14-row save/restore audit (§7; `SavedSubst` completeness; `freezeCandidates` and `tenv.seenSub` exclusions justified, Lemma 19); array-literal commitments persist on failure by design, sound by Lemma 18 under RB-1

Registry. The authoritative, current statuses are maintained in the registry (`docs/ledger.yaml`), rendered as the main paper's Assumptions and Proof Obligations tables; this paper is cited there as evidence. The registry rows it supports:

Row	Statement	Status
L-MONO	Refinement-time state evolutions preserve membership in already-installed types.	PROVED
L-CLONE	A type computed by `inter` under a cloned substitution over-approximates the intersection under the main one.	PROVED
OBL-MONOTONE	Monotone Reads Under Growth: judgements validated under ρ survive later legal transitions.	VIOLATED
OBL-RESTORE	No Commitment on Failure: failing and exploratory paths restore all relevant state.	PROVED
MR-1	No `subst.seenSub` memo recorded under `freezeCandidates` may outlive growth of an open candidate readable from its left side.	VIOLATED
CL-1	No fresh variable generated on a clone occurs in a type installed against the main substitution.	PROVED
M1	Semantic monotonicity of $[\![\cdot]\!]$ in variable envelopes (a property of the get-only, signature-opaque semantics).	ASSUMED
I-F	Fork-state form of L-INTER: the containments hold from a forked substitution state (consumed by L-CLONE's proof).	ASSUMED
RB-1	Maintenance condition: `isSub` callers discarding errors are save/restore-wrapped or unreachable by uncommitted array literals.	OPEN
SETCONSTRAINT-FUN	`TVar::setConstraint` is a functional update on a frozen value: constraints are occurrence-level, not shared state.	PROVED
E-TRANSPORT	Monotone-reads transport: an inclusion validated under ρ survives later legal transitions on the same timeline, modulo MR-1.	PROVED
TENV-EXPANDINTER	The `tenv.expandInter` mode flag is path-scoped, and both intersection forms it selects (expanded and deferred `unexpandedInter`) satisfy the L-INTER containment.	OPEN

Imports (review contract). The named statements this paper assumes from elsewhere. A reviewer treats each as an axiom, checking citations against the registry's canonical statement; no other paper's internals are in scope:

Row	Canonical statement	Owner	Status
L-UNION	`union` over-approximates set union (all levels).	B	PROVED
L-INTER	`inter` over-approximates set intersection (the containment clause).	B	PROVED
L-SUB	`Clauses::isSub` acceptance implies semantic inclusion under the current state.	D	OPEN

Summary. Three of the four rows close (L-MONO; No Commitment on Failure, with a documented carve-out; L-CLONE modulo I_f, its id-collision hazard verified unreachable and guarded by a CI assertion). The fourth — Monotone Reads — is PROVED at every audited site except its one sharp residue, MR-1, which is VIOLATED with an end-to-end witness (issue draft `issues/freeze-window-stale-memo-caches.md`); the fix (version-keying or freeze-window write suppression) must cover both `seenSub` and `seenInter`.

A Implementation Anchors

Pinned to `cf5a6bc7`.

Model	Implementation
ρ (seven fields)	`SubstState` (TTypeSubst.sk:13–49)
snapshot / reinstate	`SavedSubst`, `save`, `restore` (TTypeSubst.sk:36–74)
transitions	`Subst::grow/get` (TTypeSubst.sk:151–222)
fresh ids from version	`gen/genForField/genForLiteralCommitment` (TTypeSubst.sk:237–253)
freeze (unsaved)	`Subst.freezeCandidates` (TTypeSubst.sk:76–89); push/pop TTypeSubtyping.sk:1431–1443
clone fork	`Subst::clone` (TTypeSubst.sk:137–149)
occurrence constraints	`TVar`, `setConstraint` (TType.sk:2576–2607)
success memo	`subst.seenSub` (TTypeSubtyping.sk:1588,1591)
coinduction hypotheses	`tenv.seenSub` (TEnv.sk:74; TTypeSubtyping.sk:1603–1604); `tenv.seenInter` (TEnv.sk:69; TTypeInter.sk:154–157)
result memos	`seenInter/seenUnion/seenExpansion` (TTypeInter.sk:107–125; TTypeUnion.sk:29–32; TType.sk:650–653)
uniform-variance expansion	`expandToVectors` (TType.sk:634–678)
validation spine	`isSub_/growTVars/isCovered` (TTypeSubtyping.sk:1318–1646)
clone-compute-install	`evalVarEquality` etc. (TypeCheckRefine.sk:87–106,176–201,571–576)
main-subst refinement ops	`setLocalFilterMap` (TypeCheckRefine.sk:615); `evalIsArray/evalInstanceof` (TypeCheckRefine.sk:334,361)
form commitments	`ArrayLitTyping` (TArrayLitTyping.sk:140–148,326–432)
external grow callers	TArrayLitTyping.sk:147,352; TVariadicTyping.sk:125; TypeCheckUtils.sk:310,321,332